Operating Systems

ASSIGNMENT 1

CO1: *Apply the concepts of operating system types, structures and system calls to demonstrate their functions in various computing environments – history of operating systems, mainframe, desktop, multiprocessor, distributed, clustered, real-time and handheld systems, OS structure, services, system calls, system design, CPU states, memory hierarchy and I/O systems.*

Assignment Title: Evaluate and Justify the Operating-System Design of a Smart Hospital (Total Marks - 50)

Nature of Assignment: Individual
Duration: 1 Week
Submission Date: _____

Bloom's Levels Assessed: K5 – Evaluate (every task in this assignment is at K5 level)

Mandatory Execution and GitHub Submission

- **Every student must individually execute every task in this assignment**, including all "Complete the Code" programs.
- Create a GitHub repository named **CSA04-<RegisterNumber>-Assignment1** and share it with the course faculty (public, or private with the faculty added as a collaborator).
- Keep one folder for each task (for example: Task01, Task02 … or Q1, Q2 …), containing the C source files, shell scripts, configuration files (Dockerfile, YAML), command outputs and screenshots.
- Include a README.md with your name, register number, compiler and version (for example gcc 13), the Linux distribution and version used, the exact gcc command used to compile (for example gcc -std=c11 -Wall -Wextra -pthread), the steps to run each program, and a short note of the result of each task.
- Commit your work progressively – at least one commit for each task. The commit history will be checked; a single upload of all files on the last day will not be accepted.
- Submit the GitHub link below and on the cover page of your report. **An assignment submitted without a working GitHub link will not be evaluated.**

GitHub Repository Link: _____

How to Answer the Additional Questions

Complete the Code: Copy the given code into your repository, replace every TODO with working code without changing the rest of the program, execute it and compare your output (or result) with the expected output.

Alternative Concept and Complexity Analysis: Describe how the solution would change if the alternative concept were used, with code or pseudocode, and analyse and compare the time complexity and space complexity of both solutions using Big-O notation, supported by your own measurements wherever possible.

Scenario — Operating Systems for a Smart Hospital

A 500-bed hospital is replacing its computer systems. It needs: bedside patient monitors that must raise an alarm within 10 milliseconds of an abnormal reading; nurses' handheld tablets; doctors' desktop workstations; a central database server for patient records that must never stop; an imaging server that processes MRI scans using many processors; and a link to two partner hospitals that share records.

The IT team has proposed using one general-purpose operating system everywhere. The clinical staff are concerned that alarms may be delayed, the database may stop during updates, and the tablets' batteries will not last a shift.

The number of devices may grow:

1 ward (40 monitors, 10 tablets) → 1 hospital (500 monitors, 120 tablets, 80 workstations) → a group of 3 hospitals sharing records

You are required to evaluate the requirements of every part of the system, judge which type and structure of operating system suits each, examine how programs obtain OS services through system calls, and defend a complete operating-system plan for the hospital.

Design Challenge:

The hospital is adding remote patient monitoring at home and a cloud-based records service.

Re-evaluate your original design and determine whether it should be retained, modified or redesigned.

TASK 1 – Evaluate the Requirements of Each Computing Environment 4 Marks

Bloom's Level: K5 – Evaluate

Evaluate what each part of the hospital needs from its operating system.

Discuss:

- Response-time, reliability, throughput, battery and security requirements of each device
- Which requirements conflict (for example: fast response and many users on one machine)
- Which requirements are critical to patient safety
- How the lessons from the history of operating systems (batch, time-sharing, personal, mobile) apply to each environment

You are expected to **rank the requirements of each environment and justify the ranking**.

Alternative Concept and Complexity Analysis

If the whole hospital used a single general-purpose operating system on every device, instead of a different type of OS for each environment, what would the solution be? Evaluate the worst-case alarm response time (unbounded scheduling delay versus a guaranteed deadline), the memory needed on each device, and the administration effort.

Evidence Expected

A ranked requirement table for every environment with a justification for each ranking.

TASK 2 – Judge the Type of Operating System for Each Environment 3 Marks

Bloom's Level: K5 – Evaluate

Compare the types of operating systems for the hospital.

Discuss:

- Mainframe, desktop, multiprocessor, distributed, clustered, real-time (hard and soft) and handheld systems
- Which type best fits the monitors, tablets, workstations, database server, imaging server and partner link
- Symmetric versus asymmetric multiprocessing for the imaging server
- A clustered system with fail-over for the database server

Alternative Concept and Complexity Analysis

If the database server used a single large multiprocessor machine instead of a cluster of smaller machines with fail-over, what would the solution be? Evaluate the availability (a single point of failure versus fail-over within seconds), the cost, and how throughput grows with the number of processors (time) and the memory available (space).

Evidence Expected

A justified choice of operating-system type for each environment, with the rejected alternatives and reasons.

TASK 3 – Evaluate Operating-System Structures 4 Marks

Bloom's Level: K5 – Evaluate

Evaluate the possible structures of the operating system.

Discuss:

- Simple (monolithic), layered, microkernel, modular (loadable kernel modules) and hybrid structures
- The effect of each structure on reliability, performance and ease of updating
- Which structure suits the patient monitors and which suits the database server
- How Linux, Windows, Android and a real-time OS such as FreeRTOS or QNX are structured

Alternative Concept and Complexity Analysis

If the bedside monitors used a monolithic kernel instead of a microkernel, what would the solution be? Evaluate the time cost of a service request (one system call versus message passing between servers), the memory taken by the kernel, and the effect of a faulty driver on the whole system.

Evidence Expected

A comparison of structures against the hospital's requirements, with a defended choice for two environments.

TASK 4 – Evaluate the Services an Operating System Must Provide 4 Marks

Bloom's Level: K5 – Evaluate

Evaluate the services each environment needs.

Discuss:

- User interface, program execution, I/O operations, file systems, communication, error detection, resource allocation, accounting, protection and security
- Which services can be removed from the bedside monitor to make it smaller and faster
- Which services are essential for the database server
- How the OS protects one patient's records from another user's programs

Alternative Concept and Complexity Analysis

If the tablets used a full desktop operating system with every service enabled instead of a reduced handheld OS, what would the solution be? Evaluate the memory and storage used by the extra services (space), the start-up time and the battery life (time).

Evidence Expected

A service table for each environment with justified inclusions and exclusions.

TASK 5 – Examine System Calls — 4 Marks

Bloom's Level: K5 – Evaluate

Examine how a program obtains OS services.

Discuss:

- The categories of system calls (process control, file management, device management, information maintenance, communication, protection)
- What happens step by step when a program calls read(): library call, trap, mode switch, kernel work, return
- How parameters are passed to the kernel (registers, a block in memory, the stack)
- How strace shows the system calls made by a program

Alternative Concept and Complexity Analysis

If programs called the C library functions fopen, fread and fwrite instead of the open, read and write system calls directly, what would the solution be? Evaluate the number of system calls for a file of n bytes (about n / 4096 with library buffering versus n / b for a buffer of b bytes) and the extra buffer space used by the library.

Evidence Expected

A step-by-step explanation of a system call with strace evidence from a real program.

TASK 6 – Evaluate the Cost of System Calls — 4 Marks

Bloom's Level: K5 – Evaluate

Evaluate how the number of system calls affects performance.

Discuss:

- Why each system call costs much more than an ordinary function call
- How the buffer size changes the number of read() and write() calls when the database copies a file

- How library buffering (fread, fwrite) reduces the number of system calls
- The best buffer size for the hospital's backup program, using your measurements

Complete the Code

Complete TODO 1 below using the read() and write() system calls, and execute the program on Linux.

```
#define _POSIX_C_SOURCE 200809L
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>

/* Copy src to dst with the read() and write() system calls, using a buffer of
   bufsize bytes. Returns the number of read() calls made, or -1 on error. */
long copy_file(const char *src, const char *dst, size_t bufsize)
{
    char buf[8192];
    int in = open(src, O_RDONLY);
    if (in < 0) return -1;
    int out = open(dst, O_WRONLY | O_CREAT | O_TRUNC, 0644);
    if (out < 0) { close(in); return -1; }
    long calls = 0;
    ssize_t n;
    // TODO 1: Loop: read up to bufsize bytes into buf (count every read call in calls);
    //   stop when read returns 0 (end of file) or -1; write exactly the n bytes that
    //   were read

    close(in);
    close(out);
    return calls;
}

int main(void)
{
    FILE *f = fopen("records.dat", "w");      /* a 10,000-byte test file */
    for (int i = 0; i < 10000; i++) fputc('A' + i % 26, f);
    fclose(f);

    size_t sizes[] = {1, 512, 4096};
    for (int i = 0; i < 3; i++)
        printf("buffer %4zu bytes: %5ld read() calls\n", sizes[i],
               copy_file("records.dat", "copy.dat", sizes[i]));
    unlink("records.dat");
    unlink("copy.dat");
    return 0;
}
```

Expected output:

```
buffer    1 bytes: 10001 read() calls
buffer  512 bytes:    21 read() calls
buffer 4096 bytes:     4 read() calls
```

Run the program under strace -c and compare the number of read() calls reported with the program's output.

Alternative Concept and Complexity Analysis

If the backup program used memory-mapped files (mmap) instead of read() and write() with a buffer, what would the solution be? Evaluate the number of system calls (a few versus about n / b), the copying between kernel and user space, and the memory used for very large files.

Evidence Expected

Measured system-call counts and times for different buffer sizes, with a justified recommendation.

TASK 7 – Evaluate Process Control System Calls 4 Marks

Bloom's Level: K5 – Evaluate

Evaluate how a command shell on the database server starts and manages programs.

Discuss:

- What fork(), exec(), wait() and exit() each do
- How a shell runs a command, and how it obtains the command's exit status
- What a zombie and an orphan process are, and how the OS deals with them
- Why fork() followed by exec() is used instead of a single "create process" call

Complete the Code

Complete TODO 1 below using fork(), execlp() and waitpid(), and execute the program on Linux.

```
#define _POSIX_C_SOURCE 200809L
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main(void)
{
    printf("shell: running job 1\n");
    fflush(stdout);                          /* empty the buffer before fork() */
    // TODO 1: Create a child with fork(). In the child, replace the program with
    //   execlp("expr", "expr", "6", "*", "7", (char *)NULL) and exit(127) if exec
    //   fails. In the parent, wait for the child with waitpid() and print "job 1 exit
    //   status N" using WEXITSTATUS



    printf("shell: running job 2\n");
    fflush(stdout);
    pid = fork();
    if (pid == 0) _exit(3);                  /* a child that fails with code 3 */
    waitpid(pid, &status, 0);
    printf("job 2 exit status %d\n", WEXITSTATUS(status));
    return 0;
}
```

Expected output:

```
shell: running job 1
42
job 1 exit status 0
shell: running job 2
job 2 exit status 3
```

Explain why stdout is flushed before each fork(). Then remove the second fflush(stdout), change _exit(3) to exit(3), redirect the output to a file and explain why "shell: running job 2" now appears twice.

Alternative Concept and Complexity Analysis

If commands were run with posix_spawn() or with threads instead of fork() followed by exec(), what would the solution be? Evaluate the time to start 1,000 commands, the memory copied or shared (copy-on-write), and the isolation between commands.

Evidence Expected

A working program that runs commands like a shell, with process diagrams and a reasoned evaluation.

TASK 8 – Evaluate CPU States and Protection — 3 Marks

Bloom's Level: K5 – Evaluate

Evaluate how the hardware protects the hospital system.

Discuss:

- User mode and kernel mode, privileged instructions and the mode bit
- Interrupts, traps and the timer interrupt
- What would happen to the monitors if any program could disable interrupts
- How the hardware stops one program from reading another program's memory

Alternative Concept and Complexity Analysis

If the monitors used polling (checking the sensor in a loop) instead of interrupts, what would the solution be? Evaluate the worst-case delay before an abnormal reading is noticed (the polling interval versus the interrupt latency) and the CPU time and power wasted by polling.

Evidence Expected

A reasoned explanation of CPU modes, interrupts and protection, applied to the monitors.

TASK 9 – Evaluate the Memory Hierarchy and I/O Systems — 4 Marks

Bloom's Level: K5 – Evaluate

Evaluate how data moves between storage levels and devices.

Discuss:

- The levels of the memory hierarchy (registers, cache, main memory, SSD, hard disk, network storage) with typical access times and sizes
- Where MRI images, patient records and monitor readings should be kept
- Programmed I/O, interrupt-driven I/O and DMA, and which suits the MRI scanner
- The effect of caching on the speed of record look-up

Alternative Concept and Complexity Analysis

If the MRI scanner transferred images with interrupt-driven I/O (one interrupt per word) instead of DMA, what would the solution be? Evaluate the number of interrupts and the CPU time for a 500 MB image, and the memory buffers needed in each method.

Evidence Expected

A justified data-placement plan and I/O method for each device, using typical access times.

TASK 10 – Measure and Compare 4 Marks

Bloom's Level: K5 – Evaluate

Measure the behaviour of your programs on a Linux system.

Discuss:

- The number of system calls made by your copy program for different buffer sizes (strace -c)
- The time spent in user mode and in kernel mode (the time command)
- The time to create 1,000 processes with fork() and to create 1,000 threads
- How the measurements support or contradict your earlier judgements

Alternative Concept and Complexity Analysis

If threads were created instead of processes for the measurements, what would the result be? Evaluate the creation time for 1,000 threads versus 1,000 processes and the memory used by each, and explain the difference using what each one shares.

Evidence Expected

Measured results in tables, compared with expectations, with explained differences.

TASK 11 – Recommend and Defend the Operating-System Plan 4 Marks

Bloom's Level: K5 – Evaluate

Recommend the complete operating-system plan for the hospital.

Discuss:

- The type and structure of operating system for each environment
- The services, protection and I/O methods each will use
- The limitations and risks of your plan
- Whether the plan remains suitable for the group of three hospitals

Your recommendation must be based on **evidence from your own analysis and measurements**.

Alternative Concept and Complexity Analysis

If the whole system were moved to virtual machines on two large servers, instead of separate physical machines for each environment, what would the solution be? Evaluate the effect on response time for the monitors, the memory and processor overhead of virtualisation, and the effect on reliability.

Evidence Expected

A technically justified plan supported by analysis and measurements.

DAY-4 DESIGN CHALLENGE

TASK 12 – Re-evaluate and Adapt the Design 8 Marks

Bloom's Level: K5 – Evaluate

Changed Requirement

The hospital adds **remote monitoring devices in patients' homes** that send readings over the internet, and moves patient records to a **cloud service** shared with the partner hospitals.

Re-evaluate your original design. You may **RETAIN → MODIFY → or REDESIGN** your original solution.

A. Impact Analysis – 2 Marks

Identify which operating-system types, services and protection mechanisms are affected.

Evidence Expected

A clear analysis of how the changed requirements affect the original design.

B. Evaluation of the Original Design – 2 Marks

Evaluate whether your plan can support home devices and cloud records, considering response time, reliability and security.

Evidence Expected

A technically supported evaluation of the original design.

C. Adaptation or Redesign – 2 Marks

Describe the revised plan (for example: an embedded real-time OS on home devices, a distributed system for shared records).

Evidence Expected

An appropriate design response.

D. Evidence-Based Defence – 2 Marks

Defend your decision using evidence and the trade-offs involved.

Evidence Expected

A well-supported final decision integrating evidence and the changed requirements.

MARK DISTRIBUTION

Task	Bloom's Level	Marks
Task 1 – Requirements of Each Environment	K5 – Evaluate	4
Task 2 – Types of Operating Systems	K5 – Evaluate	3
Task 3 – Operating-System Structures	K5 – Evaluate	4
Task 4 – Operating-System Services	K5 – Evaluate	4
Task 5 – System Calls	K5 – Evaluate	4
Task 6 – Cost of System Calls	K5 – Evaluate	4
Task 7 – Process Control System Calls	K5 – Evaluate	4
Task 8 – CPU States and Protection	K5 – Evaluate	3

Task	Bloom's Level	Marks
Task 9 – Memory Hierarchy and I/O	K5 – Evaluate	4
Task 10 – Measurement and Comparison	K5 – Evaluate	4
Task 11 – Final Recommendation	K5 – Evaluate	4
Subtotal		**42**
Task 12 – Day-4 Design Challenge	K5 – Evaluate	**8**
TOTAL		**50**

Submission Requirements

- **GitHub repository link (mandatory)** containing the executed code for every task, including every completed "Complete the Code" program, script and configuration file, with input files, output screenshots and a README.md.
- Your assignment report containing the required evidence for Tasks 1–12, including requirement tables, diagrams, strace and time outputs, measurements and complexity analyses.

Assessment Verification

Students may be asked to explain, execute or modify any part of their code during a brief verification discussion/viva. The student should be able to explain and defend the work submitted as their own.

Operating Systems for a Smart Hospital:

Introduction and Final Plan at a Glance

The scenario is a 500-bed hospital that must raise a monitor alarm within 10 ms, keep a patient database running without stops, run MRI processing on many processors, keep tablets alive for a shift, and share records with two partner hospitals. The system grows in three stages: 1 ward (40 monitors, 10 tablets), 1 hospital (500 monitors, 120 tablets, 80 workstations), then a group of 3 hospitals.

Environment	OS type	Structure	Example
Bedside monitor	Hard real-time	Microkernel	QNX Neutrino (FreeRTOS on sensor chips)
Nurse tablet	Handheld / mobile	Linux kernel (modular monolithic) + managed runtime	Android Enterprise
Doctor workstation	Desktop, time-sharing	Hybrid	Windows 11 Enterprise or managed Linux desktop
Patient database	Clustered, fail-over	Modular monolithic	Linux, 2 nodes + witness
MRI imaging server	Symmetric multiprocessor	Modular monolithic	Linux, 64-core, NUMA-aware
Partner link	Distributed system	Modular monolithic	Hardened Linux gateways, mutual TLS

All programs in Tasks 6, 7 and 10 were compiled with gcc 13.3 (-O2) and run on Linux 6.18 (2 vCPUs, Intel Xeon 2.8 GHz, 4 KiB pages). Source code and raw output are in the Appendix.

Assumptions and honesty notes. Figures marked *measured* come from my runs. Figures marked *typical* are textbook or datasheet-level values, not measured here. The test machine is a virtual machine reading from the page cache, so absolute times are smaller than on a real disk; the shape of the results is what matters. Timing noise is about ±20% between runs.

Task 1: Requirements of Each Computing Environment

Each environment ranks the same five requirements differently, and the monitor's 10 ms deadline is the one requirement that no general-purpose OS can promise. Rank 1 is the most important.

Environment	Rank 1	Rank 2	Rank 3	Rank 4	Rank 5
Bedside monitor	Response time	Reliability	Security	Battery	Throughput
Nurse tablet	Battery	Security	Response time	Reliability	Throughput
Doctor workstation	Security	Response time	Reliability	Throughput	Battery
Patient database	Reliability	Security	Throughput	Response time	Battery
MRI imaging server	Throughput	Reliability	Security	Response time	Battery
Partner link	Security	Reliability	Throughput	Response time	Battery

Justification of the rankings

- **Monitor.** A late alarm is a direct patient-safety failure, so the 10 ms deadline is first. A monitor that crashes is silent, so reliability is second. Security is third because a tampered threshold or fake reading is dangerous, but the device handles little data. Battery matters only for transport and power cuts. Throughput is last: one patient produces a few hundred samples per second.
- **Tablet.** A tablet that dies mid-shift is useless, and the staff have already raised this, so battery is first. It holds patient data and is easy to lose, so security is second. Screen response (about 100 ms feels instant) is third. Reliability is fourth because a restart is an inconvenience, not a danger.
- **Workstation.** Doctors open many patient records, so protection between users comes first, then interactive speed. Battery is irrelevant on mains power.
- **Database.** The records "must never stop", so reliability is first. Records are the most sensitive asset, so security is second. Throughput (thousands of queries per second at 500 beds) is third; a query answered in 20 ms or 50 ms makes no clinical difference.
- **Imaging server.** An MRI study is hundreds of megabytes, so jobs per hour (throughput) matters most. A crash costs a re-run, not a life, so reliability is second.
- **Partner link.** It crosses a hospital boundary, so authentication and encryption come first. Delay of a second is acceptable.

Requirements that conflict

- Fast response versus many users: a time-sharing scheduler that shares the CPU fairly among 100 users cannot also promise a 10 ms deadline. This is why monitors run one fixed set of tasks.
- Throughput versus response time: large batches of MRI work keep the CPUs busy but make interactive jobs wait.
- Reliability versus updating: patching a kernel normally means a reboot, which conflicts with "never stop". Clustering resolves it by updating one node at a time.
- Battery versus responsiveness and security: background sync, encryption and a bright screen all drain the tablet.
- Security versus speed: encryption and access checks add work to every request.

Critical to patient safety: the monitor's response time and reliability, the database's availability and integrity (a wrong or missing allergy record is dangerous), and the tablet's battery during rounds. Throughput of the imaging server and battery on workstations are not safety-critical.

Lessons from OS history

Era	Lesson	Where it applies
Batch	Queue long jobs and keep the CPU busy, with no user waiting	MRI processing queue on the imaging server; nightly backups
Time-sharing	Give each user a short slice so all feel served; protect users from each other	Workstations and the shared database
Personal	One user, instant interface, low cost	Doctor workstations
Mobile	Save power, restrict apps, update over the air	Nurse tablets
Real-time (embedded)	Predictable deadlines matter more than average speed	Monitors

Alternative: one general-purpose OS on every device

If every device ran one general-purpose OS, the result would be worse on all three of the clinicians' concerns.

Measure	One general-purpose OS	Tailored OS per environment
Worst-case alarm delay	Unbounded: a fair scheduler, page faults, disk flushes and driver work can delay a task by tens of milliseconds under load (typical figure for stock Linux or Windows). No proof of 10 ms is possible.	Bounded: a fixed-priority preemptive scheduler and interrupt latency of microseconds (typical for QNX or FreeRTOS), so the 10 ms deadline can be shown by analysis
Memory per monitor	About 1–2 GB of RAM and several GB of storage (typical minimum for a desktop OS)	1–64 MB for an RTOS image plus the application
Memory across 500 monitors	About 500 × 2 GB = 1 TB of RAM bought	About 500 × 64 MB = 32 GB
Administration	One image and one patch tool for about 700 devices, which is simple. But every security flaw hits every device at once, and the database needs a reboot window	Five families to learn and patch, but a flaw in one does not reach the others, and the database is updated node by node

The single-OS plan is cheaper to learn but cannot guarantee the one number the scenario cares about most, so it is rejected.

Task 2: Type of Operating System for Each Environment

Hard real-time for monitors, handheld for tablets, desktop for workstations, clustered for the database, symmetric multiprocessing for imaging and distributed for the partner link is the best fit for each environment.

Environment	Chosen type	Why it fits	Rejected alternatives and reasons
Bedside monitor	Hard real-time	A missed 10 ms deadline is a failure, not a slow response	Soft real-time: allows occasional misses, unacceptable for alarms. Desktop/time-sharing: no deadline guarantee.
Nurse tablet	Handheld (mobile)	Aggressive power management, touch UI, app sandboxing, remote wipe	Desktop OS: too heavy for the battery (see Task 4). Real-time: no need for deadlines, poor app support.
Doctor workstation	Desktop (time-sharing, single main user)	Rich GUI, clinical applications, multitasking	Mainframe style: users do not need a terminal to a central host. Handheld: no keyboard/large screen workflow.
Patient database	Clustered with fail-over	A surviving node takes over within seconds	Single big server: single point of failure. Distributed across sites: harder consistency for the core record.
MRI imaging server	Symmetric multiprocessor	All cores run the kernel and share one memory, so image tiles are spread evenly	Asymmetric: one master core becomes a bottleneck. Cluster: image data would cross the network.
Partner link	Distributed	Separate machines in separate hospitals cooperate over a network and must tolerate link failure	Sharing one central machine: partners cannot reach it when the WAN is down, and nobody wants to give another hospital control of records.

Mainframe, desktop, multiprocessor, distributed, clustered, real-time and handheld systems.

- *Mainframe:* many users on terminals, high I/O and reliability. Its lessons (batch queues, reliability) live on in the database and imaging server, but a literal mainframe is not needed for 500 beds.
- *Desktop:* one user, responsive GUI, no deadlines.

- *Multiprocessor:* several CPUs in one machine for throughput; failure of the machine still stops everything.
- *Clustered:* several machines with shared or replicated storage and a heartbeat between them; one takes over if another fails.
- *Distributed:* machines connected by a network that may fail, each with its own memory; the OS or middleware hides the distance.
- *Real-time:* hard means a missed deadline is a failure; soft means an occasional miss only lowers quality.
- *Handheld:* limited battery, memory and screen; the OS restricts background work.

Symmetric versus asymmetric multiprocessing for the imaging server. In symmetric multiprocessing (SMP) every processor runs the same kernel and can take any task, so an MRI study cut into slices keeps all 64 cores busy and one failed core does not stop the machine. In asymmetric multiprocessing one master processor runs the OS and hands work to slaves; the master becomes the bottleneck and a single point of failure. SMP is chosen. The cost is contention on shared memory and kernel locks, which NUMA-aware scheduling reduces by keeping a slice's data near its core.

Cluster with fail-over for the database. Two database nodes plus a small witness node exchange heartbeats. If the active node stops answering, the witness and the standby agree the active node is dead, fence it (cut its storage access, so both never write), and the standby takes the virtual IP address and replays the replicated log. Fail-over takes seconds to tens of seconds (typical), which fits "must never stop" from the clinicians' view because tablets and workstations retry.

Alternative: one large multiprocessor instead of a cluster

Measure	Single large multiprocessor	Cluster of smaller machines with fail-over
Availability	Single point of failure: a board, power or OS fault stops all records until repair. Example (assumed): 2 failures a year × 4 h repair = 8 h down, 99.91%	Fail-over in about 30 s. Example (assumed): 2 failures × 30 s = 1 min a year, 99.9998%
Cost	Price rises faster than capacity for very large machines; one expensive box	Several cheaper commodity nodes, but a second copy of all storage and licences
Throughput as processors grow (time)	By Amdahl's law with 95% parallel code: 16 CPUs give 9.1×, 64 CPUs give 15.4×, the limit is 20×	Reads can be spread over replicas, but writes go to one node, so write throughput does not grow with nodes
Memory (space)	One large shared memory (a big cache is easy)	Each node has its own memory; data is stored twice

The formula is Speedup = 1 / ((1 − p) + p / N) with p = 0.95. The single machine wins on raw speed per query; the cluster wins on what the hospital values first, which is never stopping. The cluster is chosen.

Task 3: Operating-System Structures

A microkernel suits the bedside monitor because a driver crash must never silence an alarm, and a modular monolithic kernel (Linux) suits the database server because it needs raw speed and live updates.

Structure	Idea	Reliability	Performance	Ease of updating
Simple / monolithic	The whole OS is one program in kernel mode	Low: a bug in any driver can crash everything	Highest: services are plain function calls	Hard: rebuild and reboot the whole kernel
Layered	Layer n uses only layer n−1	Medium: errors are easy to locate	Lower: every request passes through several layers	Medium: replace a layer if its interface is kept
Microkernel	Kernel keeps only scheduling, memory, IPC; drivers and file system run as user processes	Highest: a failed server is restarted without a reboot	Lower: services need message passing (extra context switches)	Easy: restart or replace one server while the system runs
Modular (loadable modules)	A small core kernel loads drivers and file systems as modules on demand	Medium: modules still run in kernel mode	High: after loading, a module is a direct call	Good: load or unload a module without a reboot
Hybrid	Microkernel-style core with some services inside the kernel for speed	Medium to high	High	Good

How real systems are built

- *Linux:* monolithic kernel with loadable kernel modules (drivers, file systems, network protocols can be added at run time).
- *Windows:* hybrid. The NT executive and drivers run in kernel mode, with user-mode subsystems and services around it.
- *Android:* a Linux kernel (modular monolithic) at the bottom, with a hardware abstraction layer, a managed runtime for apps (ART) and a framework above it, so it is also layered.
- *QNX Neutrino:* a true microkernel. Drivers, file systems and network stack are separate user-mode processes that talk by message passing, and a crashed one can be restarted.
- *FreeRTOS:* a very small kernel (a few thousand lines) linked with the application into one image, with tasks, queues and timers and no memory protection by default. It is simple, but not a protected microkernel.

Defended choice for two environments

1. **Monitor: microkernel (QNX).** The alarm path is a small, high-priority process. A faulty display, network or storage driver runs outside the kernel, so it can crash and be restarted in milliseconds while the alarm process keeps its deadline. The microkernel is also small enough to analyse and certify as medical-device software.

2. **Database server: modular monolithic (Linux).** The database spends its time in system calls on storage and the network. In Linux these are direct kernel calls, which my measurements show cost about 0.25 microseconds each (Task 6). Modules let the team load a new storage-driver version without rebuilding the kernel, and the cluster hides the reboots that are still needed.

Alternative: monolithic kernel on the monitors

Measure	Monolithic kernel	Microkernel
Time for one service request	One system call. Measured on Linux: about 0.25 µs per read()/write() on small buffers (sys time ÷ calls)	A message to a server and a reply: two or more context switches, typically a few microseconds (typical figure, not measured here)
Does the difference matter?	No: both are about 1000 times smaller than the 10 ms alarm budget	No
Kernel memory	Tens of MB for a general kernel image and its resident data (typical distribution kernel)	A kernel of tens of KB, plus only the servers the monitor needs
Effect of a faulty driver	The driver runs in kernel mode with full access, so a wild pointer can corrupt the kernel and crash the machine: no alarms until reboot	The driver is an ordinary process; the kernel survives and the driver is restarted

The monolithic kernel is faster per request, but the saving is far too small to matter against a 10 ms deadline, while the crash risk matters a great deal. The microkernel stays.

Task 4: Services the Operating System Must Provide

The monitor needs only a few services done predictably, the database needs nearly all of them done strictly, and the tablet sits in between.

Service	Monitor	Tablet	Workstation	Database server	Imaging server	Partner link
User interface	Fixed display only; no shell	Touch GUI	Full GUI	Admin command line only	Command line only	None
Program execution	Fixed task set loaded at boot	Sandboxed apps	Many programs	Database processes	Batch job queue	Gateway service
I/O operations	Sensors, display, alarm output	Touch, Wi-Fi, camera	Keyboard, screen, printer	Storage and network	DMA to scanner and disks	Network

Service	Monitor	Tablet	Workstation	Database server	Imaging server	Partner link
File system	Config and small flat log	Encrypted	Full	Journaled, checksummed	Large-file, high-bandwidth	Spool queue only
Communication	One authenticated link to nurses' station	Wi-Fi	Network	Network, replication	Fast internal network	TLS to partners
Error detection	Watchdog, sensor self-test	Basic	Basic	ECC memory, checksums, heartbeat	ECC, job retry	Retry, link monitoring
Resource allocation	Static priorities, pre-allocated memory	Power-aware	Fair sharing	Memory and I/O quotas	CPU pinning, NUMA	Rate limits
Accounting	None	Battery and audit log	Login audit	Audit of every record access	Job usage	Audit of every exchange
Protection and security	Memory protection, signed firmware	App sandbox, full-disk encryption, remote wipe	User accounts, permissions	Strongest: roles, encryption, mandatory access control	Accounts, isolation	Mutual authentication

Services removed from the bedside monitor

- Graphical desktop, shell and login accounts: no one logs on to a monitor, and each adds memory and attack surface.
- General file system and printing: a small flat log is enough.
- Demand paging and swap to disk: a page fault that waits for storage can take milliseconds and breaks the 10 ms deadline, so all memory is pre-allocated.
- Dynamic program loading and accounting: the task set is fixed, so nothing needs to be installed or billed.

Removing them makes the image smaller (megabytes, not gigabytes), faster to boot and easier to verify.

Services essential for the database server: reliable I/O and a journaled file system with checksums; resource allocation that stops one query from starving others; error detection and cluster heartbeat; accounting (an audit log of who read which record); and the strongest protection.

How one patient's records are protected from another user's programs

1. *Hardware memory protection:* each process has its own address space, so one program cannot read another's memory (Task 8).
2. *Kernel-checked access:* a program reaches a file only through open(), and the kernel compares the caller's user ID and the file's permissions before it returns a descriptor.
3. *Role-based access in the database:* a nurse sees only the patients of her ward; row-level rules enforce this inside the database, not just in the application.
4. *Mandatory access control* (for example SELinux): even a compromised process is confined to its own files.
5. *Encryption at rest and in transit,* and an *audit log* so improper access is detectable.

Alternative: a full desktop OS on the tablets

Measure	Reduced handheld OS	Full desktop OS with every service
Storage (space)	A few GB for the OS (typical)	20 GB or more (typical)
Memory (space)	Idle OS uses about 1–2 GB, apps are suspended in the background	More background services and a heavier GUI, so a 4 GB tablet has little left for apps
Start-up (time)	Seconds to tens of seconds	Longer: more services start (typical)
Battery (time)	Services are paused when idle; the CPU sleeps	Indexing, updates and telemetry keep the CPU awake

An illustration with assumed numbers: a 20 Wh battery lasts 12 hours only if average power stays under 20 ÷ 12 = 1.7 W. If the extra services raise average draw from 1.4 W to 2.8 W, life falls from about 14 h to about 7 h, shorter than one shift. The reduced OS is chosen.

Task 5: How a Program Obtains OS Services (System Calls)

A program asks the kernel for a service by loading a call number and arguments into registers and executing a trap instruction; strace on a real program shows exactly those calls.

Categories of system calls

Category	Purpose	Examples (Linux)	Hospital use
Process control	Create, end, wait, allocate memory	fork, execve, wait4, exit_group, brk, mmap	Shell starts a backup job
File management	Create, open, read, write, close, delete	openat, read, write, close, unlink	Reading a patient file

Category	Purpose	Examples (Linux)	Hospital use
Device management	Request device, read/write, control	ioctl, read/write on a device file	Reading a sensor
Information maintenance	Get or set time, attributes	gettimeofday, getpid, stat	Time-stamping a reading
Communication	Messages, sockets, shared memory	socket, connect, send, recv, pipe	Monitor sends to nurses' station
Protection	Permissions and access	chmod, chown, setuid, umask	Restricting a record file

What happens step by step when a program calls read(fd, buf, count)

1. The program calls the C library function read(), an ordinary function call in user mode.
2. The library wrapper places the system-call number (0 for read on x86-64) in register rax and the arguments fd, buf and count in rdi, rsi and rdx.
3. It executes the `syscall` instruction, a trap. The hardware switches the CPU from user mode to kernel mode, saves the program counter and flags, and jumps to the fixed kernel entry address.
4. The kernel saves the remaining registers, uses rax to index its system-call table and calls the read handler.
5. The handler checks that fd is open, that the caller may read it, and that buf is a valid user address. It then copies data from the page cache into buf; if the data is not in memory it starts disk I/O and the process sleeps until the device interrupts.
1. The result (bytes read, or a negative error number) is put in rax and the kernel executes `sysret`, which restores user mode and the saved program counter.
2. The library returns the value to the program, and sets errno if it was an error.

How parameters reach the kernel: (1) in registers, which is the fastest and is what Linux uses (up to six arguments); (2) in a block (table) in memory whose address is passed in a register, used when there are more parameters than registers; (3) pushed on the stack by the program and popped by the kernel, used by some systems. The block and stack methods allow any number of parameters.

strace evidence. I compiled a small program (rd.c) that opens a file, reads it, writes it to the screen and closes it, and ran `strace ./rd`. After the dynamic loader's start-up calls, the program's own calls appear exactly as in the source:

```
openat(AT_FDCWD, "hello.txt", O_RDONLY) = 3
read(3, "patient 1017: HR=72 SpO2=98\n", 64) = 28
write(1, "patient 1017: HR=72 SpO2=98\n", 28) = 28
close(3)                                = 0
exit_group(0)                           = ?
+++ exited with 0 +++
```

Each line is: call name, decoded arguments, and the return value after the equals sign. The descriptor 3 returned by openat is the first free number after stdin, stdout and stderr (0, 1, 2); read returned 28, the number of bytes in the line, although up to 64 were requested; and write sent those 28 bytes to descriptor 1, the screen.

Alternative: fopen/fread/fwrite instead of open/read/write

I copied a 32 MiB file with `copy_stdio` (4096-byte fread buffer) under `strace -c`, and the same 1 MiB file with a tiny 16-byte user buffer.

Case	Predicted system calls	Measured read() calls	Measured write() calls
fread, 32 MiB file, 4096-byte buffer	n / 4096 = 8192	8194 (8192 + end-of-file + loader)	8193
fread, 1 MiB file, **16-byte** buffer	n / 4096 = 256	258	257
read(), 1 MiB file, 16-byte buffer (Task 6)	n / 16 = 65,536	65,538	65,537

The library collects the program's small requests in its own buffer and calls read() once per 4096 bytes, so the program asks for 16 bytes 65,536 times but the kernel is entered only 256 times, a 256-fold reduction. The cost is extra space: one 4096-byte buffer per open FILE (8 KiB for an input and an output file) and one extra copy of every byte from the library buffer to the program's buffer. With direct system calls and a buffer of b bytes, the count is n / b, so a well-chosen large b (Task 6) can beat the library's fixed 4096.

Task 6: The Cost of System Calls

Measured copy time per MiB falls 1,500-fold as the buffer grows from 1 byte to 1 MiB, almost all of it from making fewer system calls, so I recommend a 1 MiB buffer (64 KiB as the minimum acceptable).

Why a system call costs more than a function call. A function call is a jump and a return inside one mode, about 1–2 ns (typical). A system call must trap into kernel mode, save and restore registers, look up the handler, validate every argument (the kernel never trusts user pointers), run, and return, and it can disturb the CPU's caches and branch predictors. On this machine the 1-byte copy spent 0.52 s of system time on 2,097,152 calls, about 0.25 µs per call (measured), over 100 times a plain function call. Under strace each call costs about 17–18 µs because the tracer stops the program twice per call (measured with `strace -c`).

Completed code (TODO 1). `copy_sys.c` loops on `read(in, buf, bufsize)`, then calls `write(out, buf + off, n - off)` until all n bytes are written, because write() may write fewer bytes than asked. It counts both calls. The full listing is in the Appendix.

Sample output on Linux:

```
buffer=4096 bytes copied=33554432 read_calls=8192 write_calls=8192
wall=0.0339 s user=0.0115 s sys=0.0384 s
```

Measurements. Buffers of 1 and 16 bytes were tested on a 1 MiB file (a 32 MiB file at 1 byte would need 33 million calls). Larger buffers used a 32 MiB file. Time is shown per MiB so rows compare.

Buffer (bytes)	read() calls per MiB	Wall time per MiB (ms)	User time per MiB (ms)	System time per MiB (ms)
1	1,048,576	672	128	525

Buffer (bytes)	read() calls per MiB	Wall time per MiB (ms)	User time per MiB (ms)	System time per MiB (ms)
16	65,536	62	12	52
64	16,384	20.7	2.3	18.0
512	2,048	3.0	0.1	3.3
4,096	256	1.06	0.36	1.20
65,536	16	0.65	0.0	0.85
1,048,576	1	0.45	0.0	0.61
8,388,608	0.125	0.66	0.0	0.78

Repeated runs at 64 KiB gave 15.0 to 20.7 ms for the 32 MiB file (0.47–0.65 ms per MiB), so differences below about 30% are noise.

Comparing strace -c with the program's own count (1 MiB file):

Buffer	Program's read_calls	strace -c read()	strace -c write()
1	1,048,576	1,048,578	1,048,577
512	2,048	2,050	2,049
4,096	256	258	257
65,536	16	18	17
1,048,576	1	3	2

strace reports exactly 2 more read() calls and 1 more write() call than the program counted. One extra read() is the dynamic loader reading the ELF header of libc, one is the final read() that returns 0 at end of file, and the extra write() is the program's final printf output. This confirms that the program's counting is correct.

Best buffer size for the backup program. Cost falls steeply until about 64 KiB (16 calls per MiB), flattens near 1 MiB, and was somewhat slower again at 8 MiB (0.66 ms per MiB; a buffer larger than the CPU cache is a plausible cause, but I did not test this). I recommend **1 MiB**: it was fastest, uses little memory for a single backup process, and gives 1 system call per MiB. A 64 KiB buffer is within about 0.2 ms per MiB (about 7 minutes per 2 TB of data), so it is a safe choice where many backups run at once. A real disk will be slower than this page-cache test, so the buffer size matters less once the disk is the bottleneck, but a buffer under 4 KiB would still waste CPU.

Library buffering (Task 5) gives the same effect without changing the code: fread with the default 4096-byte buffer made 8,194 reads for 32 MiB, equal to the direct 4096-byte row in the table.

Alternative: memory-mapped files (mmap)

I copied the 32 MiB file with `copy_mmap` (mmap both files, one memcpy, one msync).

Measure	read()/write() with 1 MiB buffer	mmap
System calls (strace -c, 32 MiB)	About 2 × (n / b) = 64 plus a few	44 in total, including 10 mmap, 1 msync, no read/write loop
Copying between kernel and user space	Every byte is copied from the page cache to the buffer and back to the page cache	The program reads the page-cache pages directly, so the explicit copy disappears (one memcpy remains in my test because it writes into a second mapped file)
Memory for very large files	Fixed: only the buffer (1 MiB)	Address space for the whole file; pages come in on demand and are evicted by the kernel, so physical memory is bounded but page tables grow
Measured wall time, 3 runs	14–19 ms (64 KiB and 1 MiB buffers)	62–74 ms

The measurement contradicts the idea that fewer system calls always means faster. A likely cause (not measured) is page faults: 32 MiB is 8,192 pages of source and 8,192 of destination, and every first touch of a page enters the kernel much like a system call, although the kernel may map several pages per fault. For a one-pass sequential copy, read()/write() with a large buffer is about four times faster here. mmap pays off when a file is accessed randomly or many times, not for a single streaming backup.

Task 7: Process-Control System Calls (How a Shell Runs Programs)

A shell runs a command by forking a copy of itself, replacing the copy with the new program using exec, and waiting for its exit status; my program shows this working, and removing one fflush(stdout) duplicates output exactly as predicted.

The four calls

Call	What it does
fork()	Creates a child that is a copy of the caller. Returns 0 in the child and the child's PID in the parent.
exec (execlp)	Replaces the calling process's program, memory and stack with a new program. The PID stays the same; on success it never returns.
wait / waitpid	The parent sleeps until a child ends, then receives its exit status and lets the OS free the child's entry.
exit / _exit	Ends the process and passes a status (0–255) to the parent. exit() first flushes stdio buffers and runs atexit handlers; _exit() ends at once.

How the shell runs a command

```
parent (shell)                    child
--------------                    -----
printf + fflush
fork() ------------------------------> copy of the shell (new PID)
  |                               |  (set up redirections here)
  |                               |  execlp("echo", ...)  program replaced
waitpid(child) <-- sleeps               |  runs echo
  |                               |  exit(status)
  '<-- status ---------------------------'  child becomes zombie until reaped
WIFEXITED / WEXITSTATUS
```

Completed code (TODO 1). In `shell.c`, after `pid_t p1 = fork();` the child runs `execlp("echo", "echo", "hello from job 1", NULL); perror("execlp"); _exit(127);` and the parent runs `waitpid(p1, &st, 0); report(1, p1, st);`, where report() uses WIFEXITED and WEXITSTATUS to read the exit status from the status word. Job 2 is a child that simply calls _exit(3); job 3 runs a command that does not exist. Full listing in the Appendix.

Output on Linux (measured):

```
shell: running job 1
hello from job 1
shell: job 1 (pid 205) exited with status 0
shell: running job 2
shell: job 2 (pid 206) exited with status 3
shell: running job 3 (missing command)
shell: job 3 (pid 207) exited with status 127
shell: all jobs done
```

The exit status reaches the shell through waitpid: the child's exit(3) is stored by the kernel, and WEXITSTATUS returns 3. Status 127 is the shell convention for "command not found"; execlp failed, so the child reached _exit(127).

Why flush stdout before each fork(). The stdio buffer lives in the program's own memory, and fork() copies that memory. If text is still waiting in the buffer, parent and child each hold a copy and each may print it later. When stdout is a terminal it is line-buffered, so lines go out at once; when it is a file or pipe it is fully buffered, so lines wait. fflush(stdout) empties the buffer before the copy is made.

The experiment. I removed the second fflush(stdout), changed _exit(3) to exit(3), and redirected output to a file (`./shell_broken > o_bad.txt`):

```
shell: running job 1
hello from job 1
shell: job 1 (pid 214) exited with status 0
shell: running job 2
shell: job 1 (pid 214) exited with status 0
shell: running job 2
shell: job 2 (pid 215) exited with status 3
...
```

"shell: running job 2" now appears twice, and so does the "job 1 exited" line before it. After the first flush, those two lines were printed into the full buffer and not flushed. fork() copied the buffer into the child. The child's exit(3) flushes stdio buffers, so it wrote its copy (the first pair). The parent still held its own

copy and wrote it again at its next flush (the second pair). With the original _exit(3) the child would end without flushing and no duplicate would appear. (My tool's terminal was a pipe, so the same duplicates also appeared without redirection. On a real interactive terminal, line buffering hides the problem.)

Zombie and orphan processes (measured)

- *Zombie:* a child that has exited but whose parent has not yet called wait. Its entry (PID and exit status) stays in the process table. `ps` showed `PID 224 PPID 222 STAT Z zombie` while the parent slept. It uses almost no memory but occupies a process-table slot; the OS removes it when the parent calls wait() or exits.
- *Orphan:* a child whose parent exits first. My orphan program printed `child: parent before=226` and later `child: parent after=1`: the OS re-parents it to the init process (PID 1), which calls wait() and so cleans it up when it ends.

Why fork() then exec() instead of one create-process call. Between the two calls the child is still the shell's copy, so the shell can change what the new program inherits using ordinary system calls: redirect stdin or stdout to a file (`> out.txt`), connect a pipe (`a | b`), change directory, drop privileges or set limits. A single create-process call would need a parameter for each of these options. fork+exec keeps each call small, and copy-on-write makes the fork cheap.

Alternative: posix_spawn() or threads

Measured in `bench.c`, starting 1,000 commands (/bin/true) from a small parent and from a parent holding 256 MB of touched memory:

Method	Parent 1 MB: total / each	Parent 256 MB: total / each
fork() + exec	1.79 s / 1,788 µs	9.52 s / 9,524 µs
posix_spawn()	1.53 s / 1,526 µs	1.64 s / 1,638 µs
Thread (create + join, no new program)	0.05 s / 52 µs	0.06 s / 61 µs

- **Time:** posix_spawn is 15% faster for a small parent and 5.8 times faster for a large one. fork must copy the page tables of the parent's memory (256 MB is 65,536 pages), while posix_spawn does not copy the parent. For a database server with gigabytes of memory, a shell that forks directly pays this cost on every command.
- **Memory:** after fork(), parent and child share pages copy-on-write, so almost nothing is duplicated until one of them writes. posix_spawn shares nothing it does not need.
- **Isolation:** each fork/exec command is a separate process with its own address space, user ID and descriptors, so a crash or wild pointer in one command cannot touch the shell or another command. Threads share all memory and descriptors, so a crash in one command can kill the shell and every other command, and a thread cannot run a separate program at all, because exec would replace the whole process.

posix_spawn is the better choice for a command launcher with a large parent. fork+exec remains the model for a shell that must adjust descriptors between the two steps, which is why shells still use it.

Task 8: CPU States and Hardware Protection

The hardware protects the hospital system with a mode bit, interrupts and a memory-protection unit, and the monitors must use interrupts rather than polling to meet the 10 ms alarm deadline with a long battery life.

User mode and kernel mode. The CPU has a mode bit: kernel mode (0) may execute every instruction; user mode (1) may not. Privileged instructions, such as disabling interrupts, performing raw I/O, writing the memory-protection registers, changing the mode bit and halting the CPU, trap if a user program tries them. The only way into kernel mode is a system call, interrupt or exception, which jumps to a fixed address chosen by the kernel, so a program cannot run its own code in kernel mode. This is why the applications on a workstation cannot damage the database or each other.

Interrupts, traps and the timer.

Event	Cause	Example in the hospital
Interrupt	Hardware device signals the CPU asynchronously	Sensor sample ready, network packet arrived, disk transfer finished
Trap (exception)	Raised by the program itself	Divide by zero, invalid memory access, system call
Timer interrupt	Clock that fires at a fixed interval	The OS regains control from a program that never stops, and switches tasks

When an interrupt arrives the CPU saves its state, switches to kernel mode, jumps to the interrupt handler, and later returns to the interrupted program. The timer interrupt makes pre-emptive multitasking possible: even if a program loops forever, the kernel runs again within one tick.

What if any program could disable interrupts on the monitors. One faulty or malicious program could run `disable interrupts` and then: the sensor interrupt would never be serviced, so an abnormal reading would go unnoticed and no alarm would sound; the timer interrupt would stop, so the scheduler could never take the CPU back from a looping task and the alarm task would never run; and a network or disk interrupt could not be handled either. A single bug could silence a bedside monitor with no warning, which is why this instruction is privileged. As a second defence, monitors also use a hardware watchdog timer that resets the device if the software stops feeding it.

How the hardware stops one program reading another's memory. The memory-management unit (MMU) translates every address a program uses through page tables that only the kernel can edit. Each process has its own page table, so its addresses map only to its own physical pages. An address with no mapping, or a write to a read-only page, raises a trap, and the kernel ends the process (a segmentation fault). Simpler processors use a base register and a limit register: every address must lie between base and base + limit. Both schemes work because changing the registers is privileged. Small RTOS microcontrollers may only have a memory-protection unit with a few regions, which is why the monitor design relies on the microkernel to keep drivers and the alarm task in separate regions.

Alternative: polling instead of interrupts on the monitors

If the monitor checked the sensor in a loop, the worst-case time before an abnormal reading is noticed would be one polling interval T plus processing time, while with an interrupt it is the interrupt latency plus processing time.

Measure	Polling	Interrupt
Worst-case delay before the reading is noticed	T (the full interval), plus processing	Interrupt latency: a few microseconds on a typical real-time microcontroller (typical figure)
Meeting a 10 ms deadline	Needs T well below 10 ms; for example T = 1 ms leaves 9 ms for processing and transmission	Easily met: delay is about 1000 times smaller than the budget
CPU time wasted	If one poll takes 2 µs (assumed): T = 1 ms wastes 0.2% of the CPU; a continuous busy loop wastes 100%	None while idle: the CPU sleeps until the sensor interrupts
Power	A busy loop keeps the CPU fully active; even 1 kHz polling wakes it 1,000 times a second	The CPU can stay in a low-power sleep state (typically 100 to 1000 times lower power than active)
Other tasks	The loop delays display and network work, or they delay the polling	Other tasks run until the interrupt arrives

Polling can be simple and acceptable for a sensor that changes rarely and when the CPU has nothing else to do, but on a battery-backed monitor that must also display, log and communicate, it wastes energy and makes the worst-case delay depend on all the other code in the loop. Interrupts are chosen.

Task 9: Memory Hierarchy and I/O Systems

Hot patient records belong in RAM backed by NVMe SSD, MRI images in RAM during processing and then network storage, and the scanner must use DMA because interrupt-per-word I/O would take about two minutes of CPU per image.

The memory hierarchy (typical figures, approximate):

Level	Typical access time	Typical size	Cost per byte
Registers	under 1 ns	about 1 KB	Highest
Cache (L1 to L3)	1–40 ns	32 KB to tens of MB	Very high
Main memory (DRAM)	about 100 ns	GBs to 1 TB	High
SSD (NVMe)	20–100 µs	TBs	Medium

Level	Typical access time	Typical size	Cost per byte
Hard disk	5–10 ms	TBs	Low
Network storage	0.5 ms (flash) to tens of ms (disk arrays)	Hundreds of TB	Lowest

Each level down is about 10 to 1000 times slower, bigger and cheaper. The OS keeps recently used data in the faster level (caching) so most accesses are fast.

Where each kind of data should be kept

Data	Placement	Reason
Monitor readings (live)	RAM ring buffer on the monitor; short log in flash; forwarded to the database	A reading is needed in microseconds; a disk wait could break the 10 ms deadline; flash survives a power loss
Patient records	Hot working set in the database's RAM cache; table files on mirrored NVMe SSDs; replicated to the standby node; old records archived to disk-based network storage	Look-ups need microseconds to a few milliseconds; records need to survive the loss of one node
MRI images	Working data in RAM during processing; scratch on local NVMe; finished studies on network storage (disk arrays); recent thumbnails cached on SSD	A study is hundreds of MB and read mostly once; it is too big and too cold for RAM or SSD to hold for years

Programmed I/O, interrupt-driven I/O and DMA

Method	How it works	CPU involvement	Suits
Programmed I/O	CPU loops, checking the device status and copying each word	Constant: 100% busy during transfer	Very simple, tiny devices only
Interrupt-driven I/O	Device interrupts when a word or byte is ready; the handler copies it	One interrupt per unit	Slow, small transfers: monitor sensors, keyboards, touchscreens
DMA	The CPU gives the DMA controller an address and length; the controller moves the whole block to memory and interrupts once	Setup plus one interrupt per block	Large, fast transfers: MRI scanner, disks, network cards

Effect of caching on record look-up. Effective access time = hit rate × cache time + (1 − hit rate) × miss time. Using the table, a RAM hit takes 0.1 µs and a miss that goes to SSD takes 100 µs:

Cache hit rate	Average look-up time (RAM cache over SSD)	Average if misses go to a hard disk (8 ms)
0% (no cache)	100 µs	8,000 µs
90%	10.1 µs	800 µs
99%	1.1 µs	80 µs
99.9%	0.2 µs	8.1 µs

A rise from 90% to 99% hits makes look-ups about 9 times faster, which is why the database server is given a large RAM buffer cache and why the SSD, not the hard disk, sits behind it. Working-set locality (the same patients' records are read repeatedly during a ward round) is what makes high hit rates realistic.

Alternative: interrupt-driven I/O for the MRI scanner

Assume one 4-byte word per interrupt and 1 µs of CPU per interrupt (assumed; typical is 1–5 µs for entry, handler and exit). A 500 MB image is $500 \times 2^{20} = 524{,}288{,}000$ bytes.

Measure	Interrupt per word	DMA with 64 KiB blocks
Number of interrupts	524,288,000 ÷ 4 = 131,072,000	524,288,000 ÷ 65,536 = 8,000
CPU time for the interrupts	131 s (655 s at 5 µs)	8 ms
Effective transfer rate	524 MB ÷ 131 s = about 4 MB/s, limited by the CPU, far below a scanner link of about 1 GB/s	Close to the link rate: about 0.5 s for the image
Memory buffers	A word-sized device register; the CPU copies each word into the full 500 MB destination	The same 500 MB destination, or a ring of a few 64 KiB buffers if streaming to disk, plus a descriptor list of about 128 KB (8,000 × 16 bytes)

Interrupt-per-word needs 16,000 times more interrupts and would keep a core busy for over two minutes per image, during which other work, including the 64-core processing itself, would be starved. DMA is chosen for the scanner. Interrupt-driven I/O stays correct for the slow, small monitor sensors.

Task 10: Measurements and Comparison

The measurements support most of my earlier judgements, contradict one (mmap is not faster for a streaming copy), and add a new design rule: never call fork() at run time on a time-critical device.

Test system: Linux 6.18, 2 virtual CPUs, Intel Xeon 2.8 GHz, 4 KiB pages, files in the page cache. Each result is one run unless stated; repeated runs differed by up to about 30%.

Table A: system calls for the copy program (strace -c, 1 MiB file)

Buffer (bytes)	read() calls	write() calls	Total	Avg time per call under strace (µs, read / write)
1	1,048,578	1,048,577	2,097,155	17 / 18
16	65,538	65,537	131,075	17 / 18
512	2,050	2,049	4,099	19 / 21
4,096	258	257	515	20 / 26
65,536	18	17	35	32 / 41
1,048,576	3	2	5	not reported (too fast)

Calls fall in proportion to 1/b, exactly as n / b predicts. Time per call grows with larger buffers because each call copies more bytes, but total time falls because there are so many fewer calls.

Table B: user-mode and kernel-mode time (program's own getrusage, per MiB copied)

Buffer (bytes)	Wall (ms)	User (ms)	System / kernel (ms)	Kernel share of CPU time
1	672	128	525	80%
16	62	12	52	81%
512	3.0	0.1	3.3	96%
4,096	1.06	0.36	1.20	77%
65,536	0.65	0.0	0.85	about 100%
1,048,576	0.45	0.0	0.61	about 100%

User time shrinks to nothing (the program does no computation), while kernel time shrinks more slowly because the kernel must still copy every byte. The 1,048,576-byte copy spends nearly all of its time inside the kernel doing real work, not entering and leaving it. The 0.0 entries mean the user time was too small to measure reliably.

Table C: creating 1,000 processes and 1,000 threads (bench.c)

Operation	Parent with about 1 MB	Parent with 256 MB touched memory
fork() + _exit + wait, 1,000 times	183 ms (183 µs each)	7,328 ms (7.3 ms each)
pthread_create + join, 1,000 times	52 ms (52 µs each)	61 ms (61 µs each)
fork + exec /bin/true, 1,000 times	1,788 ms	9,524 ms
posix_spawn /bin/true, 1,000 times	1,526 ms	1,638 ms

Operation	Parent with about 1 MB	Parent with 256 MB touched memory
Resident memory with 1,001 threads alive (64 KiB stacks)	10.3 MB for the whole process	272 MB (256 MB data + about 16 MB)

How the measurements support or contradict my judgements

Earlier judgement	Evidence	Verdict
A system call costs far more than a function call (Task 6)	About 0.25 µs kernel time per call at 1-byte buffers, over 100 times a function call	Supported
Larger buffers reduce calls as n / b (Tasks 5, 6)	Table A matches n / b to within 3 calls	Supported
Library buffering gives n / 4096 calls (Task 5)	8,194 reads for 32 MiB with fread	Supported
Backup buffer should be 64 KiB to 1 MiB (Task 6)	Cost flattens above 64 KiB; 1 MiB fastest, 8 MiB slightly slower	Supported, with noise
mmap would be faster because it has fewer calls (my expectation before measuring)	mmap copy 62–74 ms against 14–19 ms for read/write	Contradicted: fewer calls did not mean less time for a one-pass copy
Threads are cheaper to create than processes	52 µs against 183 µs, a factor of 3.5 for a small parent	Supported
fork is cheap thanks to copy-on-write	Cheap for a small parent but 7.3 ms for a 256 MB parent, 120 times a thread	Partly contradicted: copy-on-write avoids copying data, not the page tables
posix_spawn is better for a big server (Task 7)	5.8 times faster than fork+exec with a 256 MB parent	Supported

Consequence for the design. A fork of a 256 MB process took 7.3 ms, which is 73% of the monitor's whole 10 ms alarm budget. On the monitors, all tasks must therefore be created once at start-up and never forked or spawned while patients are being watched.

Alternative: threads instead of processes

In Table C, 1,000 threads took 52 ms against 183 ms for 1,000 processes with a small parent (3.5 times faster), and 61 ms against 7,328 ms with a 256 MB parent (120 times faster). Memory: 1,001 threads together held 10.3 MB resident (under 10 KB each, plus 64 KiB of reserved stack each), and the extra

memory of 1,000 live forked processes would be their private page tables and kernel structures plus every page they write (not measured here).

The difference comes from what each shares. A thread shares its creator's address space, page tables, open files and signal handlers, so creating one needs only a new stack and a small kernel record. A forked process gets its own address space: the kernel must create a new set of page tables, copy the entries for every mapped page, mark them copy-on-write, and duplicate the file-descriptor table. The cost of fork therefore grows with the size of the parent (183 µs to 7.3 ms), while the cost of a thread does not (52 µs to 61 µs). The price of threads is no isolation: one faulty thread can corrupt the memory of all the others.

Task 11: Recommended Operating-System Plan

I recommend six purpose-matched environments, each with its own OS type, structure, services, protection and I/O method, joined by a secured network; the monitors work alone if every other system fails.

Environment	Type and structure	Services kept	Protection	I/O method
Bedside monitor	Hard real-time, microkernel (QNX; FreeRTOS on sensor chips)	Fixed task set, sensor and alarm I/O, watchdog, one authenticated network link	Memory protection between alarm task and drivers, signed firmware, no user accounts, interrupts privileged	Interrupt-driven sensors; no polling; no paging
Nurse tablet	Handheld, Linux kernel with managed runtime (Android Enterprise)	Touch UI, sandboxed apps, Wi-Fi, power management	App sandbox, full-disk encryption, device management with remote wipe	Interrupt-driven plus DMA
Doctor workstation	Desktop time-sharing, hybrid (Windows) or managed Linux	Full GUI, multitasking, printing, audit	Per-user accounts, permissions, disk encryption, central patching	Interrupt-driven plus DMA
Patient database	Clustered with fail-over; modular monolithic Linux	Journaled file system, replication, audit log, resource quotas	Role-based and row-level access, mandatory access control, encryption	DMA to mirrored NVMe; 1 MiB buffers for backup
Imaging server	Symmetric multiprocessor, modular monolithic Linux, 64 cores	Batch job queue, large-file storage, NUMA scheduling	Accounts, per-study permissions, isolated job users	DMA from the scanner

Environment	Type and structure	Services kept	Protection	I/O method
Partner link	Distributed system, hardened Linux gateways	Message queue, retries, TLS	Mutual TLS, audit of every exchange, minimal services	Interrupt-driven plus DMA network cards

Evidence from my analysis and measurements that drives the plan

1. *Monitors use a hard real-time microkernel.* A general-purpose scheduler cannot bound alarm delay (Task 1); a driver fault in a monolithic kernel can stop all alarms (Task 3); fork of a 256 MB process took 7.3 ms, 73% of the 10 ms budget (Task 10), so tasks are created once at boot.
2. *The database uses Linux in a cluster.* Direct kernel calls cost about 0.25 µs each, and a two-node cluster cuts expected downtime from hours to about a minute a year in my example (Task 2). Raw speed per CPU is less important than never stopping.
3. *Backup and bulk copy use a 1 MiB buffer.* It made 1 system call per MiB and was the fastest of eight sizes tested (Task 6). The nightly job should use read()/write(), not mmap, which was about four times slower for a one-pass copy.
4. *Large servers launch commands with posix_spawn.* It was 5.8 times faster than fork+exec from a 256 MB process (Task 7).
5. *The imaging server uses SMP and DMA.* Interrupt-per-word transfer of one 500 MB image would need 131 million interrupts and over two minutes of CPU (Task 9).

Limitations and risks

- *Five OS families* mean more skills, more patch streams and more tools to learn. Mitigation: standardise on Linux for all servers (three of the six environments), and one tablet and one workstation image.
- *Split-brain in the cluster:* if the heartbeat link fails, both nodes may think they are the primary and corrupt data. Mitigation: a witness node and fencing.
- *Real-time guarantees can be broken by one badly written task* or a priority inversion in the monitor. Mitigation: static analysis of worst-case execution time, priority inheritance, a watchdog.
- *Fail-over is not instant:* about 30 s of unavailability (assumed) per switch, so clients must retry. The monitors must keep alarming locally during that time.
- *My test machine was virtual and cached,* so Task 6 and 10 numbers show shape, not absolute hospital performance. The buffer size should be re-measured on the production storage.
- *Security surface:* the more devices and links, the more attack surface; every network link needs authentication and encryption.

Does the plan suit a group of three hospitals? Yes, with changes at the shared layers. Assuming equal sizes, the group has about 1,500 monitors, 360 tablets and 240 workstations. Monitors scale freely because each raises its own alarm locally and only sends data onward. Tablets and workstations scale by adding managed images. The database is the limit, because writes go to one node and Amdahl's law caps a single machine at 20 times for 95% parallel code: it should be partitioned by hospital, with a cluster per site and asynchronous replication of shared records between sites. WAN delay of tens of milliseconds

means the partner link must never sit in the alarm path. The imaging servers scale by adding one per hospital.

Alternative: virtual machines on two large servers

Measure	Separate physical machines	Everything as VMs on two large servers
Response time for monitors	Direct hardware access, bounded latency	The bedside device itself still has to be physical. If its processing were virtualised, the hypervisor adds delay on every interrupt (tens of microseconds, typical), and if virtual CPUs share physical cores, a VM can wait milliseconds for its turn, so the 10 ms guarantee is lost unless cores are pinned.
Memory and CPU overhead	None beyond the OS	Each VM has its own guest OS (hundreds of MB to GB), and the hypervisor takes memory and CPU; typical CPU overhead 2–10% (typical, not measured). More VMs per host means more total overhead.
Reliability	A failure affects one environment	A host failure takes down every VM on it. The two hosts form one failure domain (same rack, power, hypervisor bug). Restart on the other host takes tens of seconds to minutes, which is slower than database fail-over and needs the other host to have spare capacity.
Administration	Many machines	Fewer machines, snapshots and live migration make patching easier

Virtual machines are a good fit for workstation desktops, test systems and the partner gateway, where consolidation saves hardware and a short restart is acceptable. They are rejected for the monitor alarm path and for the two database nodes, which must sit on separate physical hosts so that one failure cannot take out both.

Task 12: Day-4 Design Challenge: Re-evaluate and Adapt the Design

Decision: MODIFY. Keep the in-hospital design, whose requirements have not changed, and add a new real-time home-device tier and a distributed cloud records layer, because the original plan assumed a trusted hospital network and cannot meet a 10 ms alarm over the internet.

A. Impact Analysis

Area	Original design	Affected by home devices and cloud?	Impact
OS types	Six environments, all on the hospital network	Yes: two new environments	New: embedded hard real-time OS on home devices; distributed / cloud (virtualised) OS for shared records. The partner link changes from a two-partner gateway to a cloud gateway.
Bedside monitor, tablet, workstation, imaging	Hospital LAN only	Unchanged in OS type	Tablets and workstations gain a new cloud connection; monitors stay local.
Patient database cluster	Sole system of record	Yes	Becomes one node of a larger system and must stay consistent with the cloud copy.
Services	Communication was a LAN link	Yes	Added: secure internet communication, store-and-forward file storage, over-the-air update, offline detection, cross-organisation audit.
Protection	Trust boundary is the hospital network	Yes, strongly	Devices sit in patients' homes and are physically reachable. Every device and user needs its own identity; data must be encrypted in transit and at rest; the cloud is shared with other organisations.
Response time	LAN delay under 1 ms	Yes	Internet round trip is typically 30–150 ms with jitter and loss (typical), far above 10 ms.
Reliability	UPS power, IT staff, redundant network	Yes	Home power, Wi-Fi and broadband fail with no technician nearby.

B. Evaluation of the Original Design

Requirement	Can the original plan support it?	Reason
Response time	Partly	The QNX monitor design keeps the 10 ms alarm locally, but a remote alarm cannot be guaranteed at 10 ms through the internet. The requirement must be

Requirement	Can the original plan support it?	Reason
		redefined: local alarm in 10 ms, remote notification in seconds.
Reliability	No, not as designed	The cluster fail-over covers a node failure inside one building, not a lost internet link, a regional cloud outage or a power cut in a home. A monitor that only sends readings and alarms remotely would be silent in an outage.
Security	No	The original had no per-device identity, signed updates or encryption on public networks, and its protection assumed the hospital perimeter. Home devices can be stolen, opened, or attacked from the internet; the cloud adds multi-tenant risk.
Scale	Partly	Two partner links are manageable; many hospitals and thousands of homes need automated onboarding and updating.
Records sharing	Partly	One database cluster is a single system of record with no consistent copy outside the building.

The design's core choices (hard real-time microkernel for alarms, a clustered Linux database, SMP imaging) remain valid. The gaps are at the edges: the network boundary, the shared records and the new device type.

C. Adaptation: the Revised Plan

```
HOME                    INTERNET              CLOUD (3 zones)          HOSPITAL
----                    --------              ---------------          --------
Home monitor  --mTLS/MQTT-->  gateway  --> device     --> replicated records  <--> database cluster
(RTOS, local alarm,                    registry       store (majority        (local system of
 flash buffer,                  + audit       quorum, 3 replicas)     record, keeps working
 signed OTA)                                    ^                      if WAN is down)
                                                |                tablets, workstations
                                    partner hospitals (own keys,    monitors (unchanged)
                                    role-based access)
```

- **Home device: embedded hard real-time OS** (FreeRTOS or Zephyr on a microcontroller, QNX on a larger device). The abnormal-reading check and the local alarm run on the device, interrupt-driven, with the same 10 ms deadline as the bedside monitor. Services are cut to the minimum: no shell, no open ports, no user accounts.
- **Store-and-forward.** At one 64-byte reading per second, 24 hours is 86,400 × 64 = 5.5 MB, so 72 hours (16.6 MB) fits easily in flash. If the link drops, readings queue and are sent in order when it returns. A heartbeat every 30 s lets the cloud raise a "device offline" alert, so silence is never mistaken for health.

- **Protection.** Secure boot, a per-device certificate, mutual TLS, encrypted storage, and signed over-the-air updates written to a second flash partition, so a failed update falls back to the old image instead of bricking the device.
- **Cloud records: a distributed system.** Records are stored on a replicated store across three availability zones that needs a majority (2 of 3) to commit writes. If each replica were up 99.9% of the time and failures were independent (assumed), the store is available $1 - (3 \times 0.001^2 \times 0.999 + 0.001^3) = 99.9997\%$, about 1.6 minutes of downtime a year. Each hospital's data is isolated by tenant, encrypted with keys the hospital controls, and every access is logged.
- **Edge-first hospital.** The hospital's own database cluster remains the system of record for care inside the hospital, so patient care continues if the WAN or cloud is down; changes are synchronised to the cloud through a message queue and reconciled with version numbers.
- **Tablets and workstations** keep their OS types; they gain device certificates and an identity-aware access policy (no trust based on network location).

D. Evidence-Based Defence

Decision	Evidence	Trade-off accepted
Local alarm on the home device rather than cloud processing	Internet round trip of 30–150 ms (typical) is 3 to 15 times the 10 ms budget; my Task 8 analysis shows interrupts give microsecond detection	Each home device needs a capable processor, certified firmware and a way to update it
Retain the real-time microkernel and the exclusion of fork at run time	A 256 MB fork took 7.3 ms, 73% of the budget (Task 10)	None new
Keep the hospital cluster as the local system of record, with cloud sync	Fail-over example: about 1 minute a year against 8 hours for a single machine (Task 2); a cloud outage must not stop care	Two copies need reconciling; conflicts must be resolved by rules
Cloud replicas with majority quorum across three zones	99.9997% under stated assumptions against 99.9% for a single replica	Cross-site commits cost network round trips of 10–50 ms, so cloud writes are slower than local ones; recurring hosting cost
Per-device certificates and signed updates	Home devices are physically reachable and internet-facing	Key management, certificate renewal and onboarding of thousands of devices need automation and staff
Batch readings in each message	Library buffering cut system calls 256-fold (Task 5); every transmission also costs radio energy on a battery device	Alerts must bypass batching so that an alarm is sent at once

Decision	Evidence	Trade-off accepted
Do not redesign the whole system	The in-hospital requirements and my measurements are unchanged	The five-OS-family skills burden grows to six

A full redesign (everything in the cloud) was rejected because it would put the 10 ms alarm and hospital care behind an internet link. Retaining the original design unchanged was rejected because it fails the reliability and security checks in part B. **Modifying** it, by keeping what the evidence still supports and adding real-time edge devices, a distributed cloud store and zero-trust protection, meets the changed requirements with the least risk.

Remaining risks: a cloud-provider or key-management failure (mitigated by the local system of record and hospital-held keys), an unreliable over-the-air update (mitigated by dual partitions and staged roll-out), and the privacy rules that apply to shared records, which need legal review beyond this assignment.

Appendix: Source Code and How to Reproduce

Compile and run on any Linux machine with gcc and strace installed:

```
gcc -O2 -Wall -o copy_sys copy_sys.c
gcc -O2 -Wall -o shell shell.c
gcc -O2 -Wall -pthread -o bench bench.c
head -c 32M /dev/urandom > in32.bin
./copy_sys in32.bin out.bin 65536
strace -c ./copy_sys in1m.bin out.bin 4096
./shell > out.txt
./bench 0      # parent of about 1 MB
./bench 256    # parent with 256 MB of touched memory
```

copy_sys.c (Task 6, TODO 1 completed)

```
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <unistd.h>
#include <time.h>
#include <sys/resource.h>

int main(int argc, char *argv[]) {
   if (argc != 4) { fprintf(stderr, "usage: %s in out bufsize\n", argv[0]); return 1; }
   size_t bufsize = (size_t)atol(argv[3]);
   char *buf = malloc(bufsize);
   int in = open(argv[1], O_RDONLY);
   int out = open(argv[2], O_WRONLY | O_CREAT | O_TRUNC, 0644);
   if (in < 0 || out < 0 || !buf) { perror("setup"); return 1; }
   long reads = 0, writes = 0; long long total = 0;
   struct timespec t0, t1; clock_gettime(CLOCK_MONOTONIC, &t0);
   ssize_t n;
   /* TODO 1 (completed) */
   while ((n = read(in, buf, bufsize)) > 0) {
```

```
        reads++;
        ssize_t off = 0;
        while (off < n) {                   /* write() may be partial */
            ssize_t w = write(out, buf + off, n - off);
            if (w < 0) { perror("write"); return 1; }
            off += w; writes++;
        }
        total += n;
    }
    clock_gettime(CLOCK_MONOTONIC, &t1);
    close(in); close(out);
    struct rusage ru; getrusage(RUSAGE_SELF, &ru);
    double wall = (t1.tv_sec - t0.tv_sec) + (t1.tv_nsec - t0.tv_nsec) / 1e9;
    printf("buffer=%zu bytes copied=%lld read_calls=%ld write_calls=%ld\n", bufsize, total, reads, writes);
    printf("wall=%.4f s user=%.4f s sys=%.4f s\n", wall,
           ru.ru_utime.tv_sec + ru.ru_utime.tv_usec / 1e6,
           ru.ru_stime.tv_sec + ru.ru_stime.tv_usec / 1e6);
    return 0;
}
```

shell.c (Task 7, TODO 1 completed)

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

static void report(int job, pid_t pid, int st) {
    if (WIFEXITED(st)) printf("shell: job %d (pid %d) exited with status %d\n", job, pid, WEXITSTATUS(st));
    else if (WIFSIGNALED(st)) printf("shell: job %d killed by signal %d\n", job, WTERMSIG(st));
}
int main(void) {
    int st;
    printf("shell: running job 1\n");
    fflush(stdout);                         /* first fflush */
    pid_t p1 = fork();
    if (p1 == 0) {                          /* TODO 1 (completed) */
        execlp("echo", "echo", "hello from job 1", (char *)NULL);
        perror("execlp"); _exit(127);
    }
    waitpid(p1, &st, 0); report(1, p1, st);

    printf("shell: running job 2\n");
    fflush(stdout);                         /* second fflush */
    pid_t p2 = fork();
    if (p2 == 0) { _exit(3); }              /* job 2 exits with 3 */
    waitpid(p2, &st, 0); report(2, p2, st);

    printf("shell: running job 3 (missing command)\n");
    fflush(stdout);
```

```
    pid_t p3 = fork();
    if (p3 == 0) { execlp("no_such_cmd", "no_such_cmd", (char *)NULL); _exit(127); }
    waitpid(p3, &st, 0); report(3, p3, st);
    printf("shell: all jobs done\n");
    return 0;
}
```

The experiment version `shell_broken.c` is the same file with the second `fflush(stdout)` deleted and `_exit(3)` changed to `exit(3)`.

bench.c (Tasks 7 and 10)

```
#define _GNU_SOURCE
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <pthread.h>
#include <spawn.h>
#include <sys/wait.h>
#include <time.h>
extern char **environ;
static double now(void){ struct timespec t; clock_gettime(CLOCK_MONOTONIC,&t); return
t.tv_sec+t.tv_nsec/1e9; }
static void *thr(void *a){ return a; }
int main(int argc, char **argv) {
    int N = 1000; size_t mb = argc > 1 ? atol(argv[1]) : 0;
    char *big = NULL;
    if (mb) { big = malloc(mb << 20); memset(big, 1, mb << 20); }  /* memory the parent owns */
    double t = now();
    for (int i = 0; i < N; i++) { pid_t p = fork(); if (p == 0) _exit(0); waitpid(p, 0, 0); }
    printf("fork+_exit+wait x%d : %.1f ms\n", N, (now()-t)*1e3);
    t = now();
    for (int i = 0; i < N; i++) { pthread_t th; pthread_create(&th, 0, thr, 0); pthread_join(th, 0); }
    printf("pthread create+join x%d : %.1f ms\n", N, (now()-t)*1e3);
    t = now();
    for (int i = 0; i < N; i++) { pid_t p = fork(); if (p == 0) { execl("/bin/true","true",(char*)0); _exit(127);}
waitpid(p,0,0); }
    printf("fork+exec /bin/true x%d : %.1f ms\n", N, (now()-t)*1e3);
    t = now();
    for (int i = 0; i < N; i++) { pid_t p; char *a[]={"true",0}; posix_spawn(&p,"/bin/true",0,0,a,environ);
waitpid(p,0,0); }
    printf("posix_spawn /bin/true x%d : %.1f ms\n", N, (now()-t)*1e3);
    return 0;
}
```

The full bench.c used for the results also keeps 1,000 threads alive at once with 64 KiB stacks and prints VmRSS from /proc/self/status. The other programs (copy_stdio.c using fread/fwrite, copy_mmap.c using mmap and memcpy, zombie.c, orphan.c, and the 10-line rd.c used for the strace trace in Task 5) follow the same pattern as above.

Limits of the evidence. All timings come from one virtual machine with files in the page cache. Figures labelled "typical" or "assumed" in the text are standard values or stated assumptions, not measurements.